

Indian Statistical Institute

Kolkata 700108, India

Hallucination Detection in Large Language Models Using Internal Representations

DISSERTATION SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology

in

Computer Science (with specialization in Data Science)

by

Chinmoy Sahoo

[Roll No: CS2412]

under the supervision of

Prof. Ujjwal Bhattacharya

Professor

Computer Vision and Pattern Recognition Unit

Indian Statistical Institute

Kolkata 700108, India

June 2026

Certificate

This is to certify that the dissertation entitled “**Hallucination Detection in Large Language Models Using Internal Representations**” submitted by **Chinmoy Sahoo** (Roll No: CS2412) to the Indian Statistical Institute, Kolkata, in partial fulfilment of the requirements for the award of the degree of **Master of Technology in Computer Science (with specialization in Data Science)** is a bona fide record of work carried out by him under my supervision and guidance. The dissertation has fulfilled all the requirements as per the regulations of this institute and, in my opinion, has reached the standard needed for submission.

Prof. Ujjwal Bhattacharya

Professor,
Computer Vision and Pattern Recognition Unit,
Indian Statistical Institute,
Kolkata 700108, India.

Declaration

I hereby declare that the work presented in this dissertation, entitled “**Hallucination Detection in Large Language Models Using Internal Representations**”, is the result of my own investigation carried out under the supervision of Prof. Ujjwal Bhattacharya at the Computer Vision and Pattern Recognition Unit, Indian Statistical Institute, Kolkata. All external sources of information, ideas and figures have been duly acknowledged and cited. To the best of my knowledge, this work has not been submitted, in whole or in part, for any other degree or diploma.

Chinmoy Sahoo

Roll No: CS2412

Indian Statistical Institute,
Kolkata 700108, India.

*To my parents and my teachers,
for their patience and their faith in slow, careful work.*

Acknowledgements

I owe my deepest gratitude to my supervisor, Prof. Ujjwal Bhattacharya, of the Computer Vision and Pattern Recognition Unit at the Indian Statistical Institute, Kolkata. His steady guidance, his insistence on careful claims over comfortable ones, and his encouragement to keep the experimental scope honest shaped this dissertation at every stage. I am thankful to the faculty and research staff of the Institute for building an environment in which ideas can be tested rather than merely asserted, and for the computational guidance that made a free-tier study of this kind feasible. I thank my fellow students for many useful discussions on language models, evaluation, and the practical difficulties of running large models on modest hardware. Finally, I thank my family for their unwavering support throughout this work.

Chinmoy Sahoo

Indian Statistical Institute,
Kolkata 700108, India.

Abstract

Large language models write fluent prose that is, often enough to matter, confidently wrong. This failure — hallucination — is the main barrier to using these models where correctness counts. One family of detectors reads a model’s own hidden activations as it writes and learns to separate truthful continuations from fabricated ones, with no external retrieval and no repeated sampling. Such detectors are cheap to run, yet the published evidence for them is spread thinly across different models, datasets, and hardware budgets, which makes it hard to say which ingredient actually carries the signal.

This dissertation settles that question with one reproducible study that runs end to end on free cloud GPUs. Building on the label-free data generation of the MIND framework — where a model continues a truncated encyclopedia article and is labelled by whether it recovers the original entity — we design three inexpensive signals read from a single forward pass: how far the representation travels from one layer to the next, how widely it spreads across the layer stack, and how uncertain the model is as it writes. Around these we run a systematic ablation of **sixteen feature configurations** — the bare hidden state, each new signal alone and together, nine features taken from the recent literature, six further exploratory signals, and their combinations. Every configuration shares one classifier, one data split, and one set of scoring rows, and each is measured against **six detectors** — four state-of-the-art methods and two simple references — across as many as ten question-answering, summarisation, and dialogue benchmarks, on several open models ranging from one to seven billion parameters.

The study separates two regimes that prior work tends to blur: detecting hallucinations in text of the same kind the detector was trained on, and transferring to unseen task types. In-distribution detection is reliable; transfer is the hard part, and there the outcome is genuinely model-dependent. On one six-billion-parameter model a supervised mid-layer probe is strongest while our richer configurations sit just behind it; on a second model of the same scale a configuration built from the exploratory internal-state signals is the strongest method of all, ahead of every baseline. The detectors also differ sharply in cost: every single-pass method, ours and most baselines alike, is far cheaper

at inference than the sampling-based detector, which repeats generation several times per input.

The contribution is therefore a careful, free-tier-reproducible map of which internal-state signals are worth their cost and where they fail — the controlled, like-for-like comparison this corner of the field has lacked — rather than a single headline number. Along the way the work delivers a reusable pipeline, a clean per-configuration and per-baseline result set on two open models, and an honest

Contents

Certificate	i
Declaration	ii
Acknowledgements	iv
Abstract	v
List of Figures	x
List of Tables	xii
List of Algorithms	xiv
List of Abbreviations	xv
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Research Objectives	3
1.4 Thesis Contributions	3
1.5 Thesis Organization	4
2 Background	5
2.1 Transformer language models and their hidden states	5
2.2 What counts as a hallucination	6
2.3 Detection as a classification problem	6
2.4 How detection is measured	7
3 Related Work	8
3.1 Why surface-overlap metrics are not enough	8
3.2 Family I: self-evaluation and calibration	8

3.3	Family II: sampling and consistency	9
3.4	Family III: internal-state probing	10
3.4.1	SAPLMA: hidden states reveal untruths	10
3.4.2	MIND: automatic labels and real-time detection	10
3.4.3	HaloScope: labels from a latent subspace	10
3.4.4	HalluShift: multi-layer fusion with probability features	11
3.4.5	Recent developments	12
3.5	Family IV: retrieval-based fact verification	12
3.6	Benchmarks and metrics used in this area	13
3.7	Open problems	13
3.8	Positioning of this dissertation	14
4	Methodology	15
4.1	Overall pipeline	15
4.2	Automatic label generation	16
4.3	The canonical feature	18
4.4	Three internal-state features	18
4.5	Two further feature families	19
4.5.1	Literature features	19
4.5.2	Additional exploratory features	20
4.6	Feature configurations	21
4.7	The classifier	22
4.8	Evaluation protocol	22
4.9	Baselines	23
4.10	Models and free-hardware implementation	24
5	Results and Discussion	25
5.1	What has been measured	25
5.2	In-distribution detection	25
5.3	Transfer results on the fully measured model	26
5.4	One benchmark dominates the averages	27
5.5	The second model: a proposed configuration leads	28
5.6	Models still in progress	29
5.7	Inference cost	30
5.8	Discussion	31
6	Conclusion and Future Work	34
6.1	Summary	34

CONTENTS

6.2	Limitations	35
6.3	Future work	35
6.4	Closing remark	36
	References	37
	A Full Result Tables	40
	B Hyperparameters and Implementation Details	42
	C Reproducibility Notes	44

List of Figures

1.1	The practical case for internal-state probing. A consistency check that draws roughly twenty extra samples per response costs orders of magnitude more time than a classifier that reads activations the model has already produced. Latencies are indicative figures from the internal-state detection literature, plotted on a logarithmic scale.	2
3.1	Four families of hallucination detection. This dissertation works in the internal-state probing family (outlined), which reads a model’s own activations and is the cheapest to run at inference time.	9
4.1	The augmented internal-state detector. Every signal is read from one forward pass of a frozen model, so detection adds negligible cost over normal generation.	16
4.2	Automatic, label-free data generation. The model’s own first-token prediction decides the label; no human annotation is used, and the procedure is re-run separately for each target model.	17
4.3	The feature families that make up the configurations. Each configuration is the canonical backbone plus a fixed concatenation of chosen scalar families; there is no automatic feature selection.	21
5.1	In-distribution detection is easier than cross-task transfer. The two left bars are earlier mid-term in-distribution figures at a larger data scale; the rest are from the unified runs. Transfer means are computed over the ten-benchmark suite.	27
5.2	Mean transfer AUROC of the sixteen configurations on the six-billion model. The three-feature internal-state set (highlighted) gives only a small change over the canonical backbone; the gains come from the richer stacks (highlighted).	29

5.3	The proposed configurations against the baselines on the six-billion model. The best configurations are competitive with the field but do not overtake the strongest baseline, the supervised mid-layer probe.	30
5.4	Per-benchmark transfer AUROC on the six-billion model, configurations above the line and baselines below. The HaluEval-QA column stands out; elsewhere most cells are near 0.5.	31
5.5	The same data as lines. The spike on HaluEval-QA is what separates the leading methods; the other benchmarks cluster near chance.	32
5.6	Estimated per-text detection latency on the six-billion model (single GPU), derived from the measured feature-extraction stage timings. Single-pass methods are dominated by one forward pass and the classifier adds under a millisecond; the sampling-based baseline repeats generation and is several times slower. The instrumented notebooks record exact per-method timings when run.	33

List of Tables

3.1	Design separation between HalluShift [1] and the detector developed in this dissertation. The two share only the broad premise of internal-state probing.	12
3.2	Benchmark coverage of representative prior methods. A check mark indicates the method reports results on that benchmark in its own paper. This dissertation evaluates all configurations on the full set.	13
4.1	The sixteen feature configurations. The input dimension is shown for the six-billion-parameter model (hidden width $d = 4096$); for other models the canonical part changes with d while the scalar counts stay the same.	22
4.2	Baselines re-implemented inside the unified pipeline. “Dim” is the dimensionality of each method’s feature or score for the six-billion-parameter model.	24
5.1	Status of each model at the time of writing. Only the model with a complete, file-backed set of measurements is used for the headline comparison.	26
5.2	Six-billion-parameter model: the sixteen feature configurations, by mean transfer AUROC over ten benchmarks, with the in-distribution figure for reference. Higher is better; 0.5 is chance.	28
5.3	Six-billion-parameter model: the six baselines, by mean transfer AUROC over ten benchmarks. The strongest method overall is the supervised mid-layer probe.	29
5.4	Second six-billion-class model (OPT): the four exploratory-feature configurations and the six baselines, by mean transfer AUROC over ten benchmarks. The proposed confidence-trajectory configuration leads every baseline. Higher is better; 0.5 is chance.	32

A.1	Full per-benchmark transfer AUROC on the six-billion-parameter model (GPT-J): sixteen feature configurations and six baselines, scored on identical rows of all ten benchmarks. Bold marks the best value in each column. Source: the consolidated configuration and baseline measurement files.	41
A.2	Diagnostic configuration run on the second six-billion-class model (OPT-6.7B), the part backed by a measurement file in the repository: twelve configurations on the original seven-benchmark suite at a small number of rows per benchmark. At this sample size the numbers are diagnostic, not scientific. AUROC; 0.5 is chance.	41

List of Algorithms

1	Automatic label generation for one article	17
2	Single-pass feature extraction and multi-task evaluation	23

List of Abbreviations

Abbrev.	Expansion
LLM	Large Language Model
MIND	The internal-state hallucination-detection framework of Su et al. (2024) that this work extends; used as a proper name
MLP	Multi-Layer Perceptron
AUROC	Area Under the Receiver-Operating-Characteristic curve
ROC	Receiver Operating Characteristic
ECE	Expected Calibration Error
QA	Question Answering
NLI	Natural Language Inference
JSD	Jensen–Shannon Divergence
SOTA	State Of The Art
BLEURT	Bilingual Evaluation Understudy with Representations from Transformers (a learned text-similarity metric)
VRAM	Video Random-Access Memory (GPU memory)
bf16	The bfloat16 16-bit floating-point number format
HF	Hugging Face (model and dataset hub)
NLL	Negative Log-Likelihood
SVD	Singular Value Decomposition

Each abbreviation is also defined in words at its first use in the chapter where it first appears.

Chapter 1

Introduction

1.1 Motivation

A large language model, hereafter abbreviated LLM, is a neural network trained to predict the next token in a sequence of text. Once trained on enough data, such a model can answer questions, summarise documents, and hold a conversation. The same property that makes these systems useful also makes them dangerous in a specific way: they are optimised to produce text that *looks* right, not text that *is* right. As a result, an LLM will sometimes state a wrong fact with the same fluency and confidence that it uses for a correct one. This behaviour is widely called *hallucination*, and a now-standard survey of the topic describes it as the generation of content that is fluent and plausible yet unfaithful to the source or to the world [2].

Hallucination is not a rare glitch. It is a structural consequence of how these models are built, and it appears across every task to which LLMs are applied. When a model invents a citation, misstates a medical dosage, or confidently answers a question whose premise is false, the cost is borne by whoever trusted the answer. As LLMs move from demonstrations into tools that professionals rely on, the ability to attach a warning to an unreliable answer becomes as important as the answer itself. The problem this dissertation addresses is therefore not how to stop a model from hallucinating, but how to *detect*, cheaply and in real time, when it probably has.

Several families of detectors exist, and we review them in detail in Chapter 3. One popular family samples the model many times and checks whether the answers agree, on the intuition that fabricated content is unstable while real knowledge is consistent; the best-known method of this kind is SelfCheckGPT [3]. This approach works without any access to the model’s internals, but it is expensive: it requires many full generations for every single answer it checks. A second family takes the opposite view. It assumes we can open the model up and read the hidden activations it produces while generating, and it learns a small classifier that maps those activations to a hallucination probability. The appeal is cost. Because the activations are already computed during normal generation, reading them adds almost nothing to the running time.

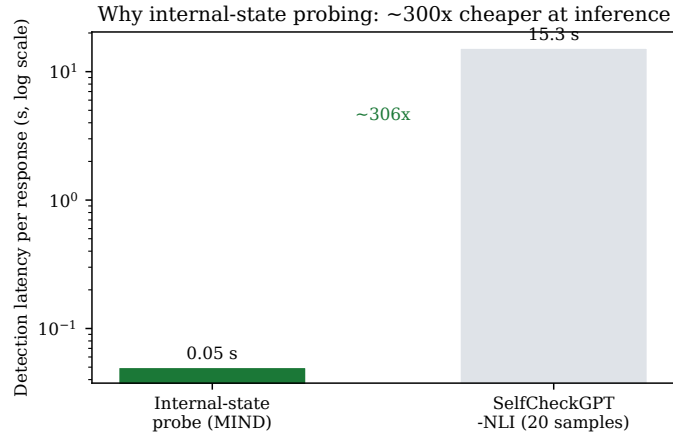


Figure 1.1: The practical case for internal-state probing. A consistency check that draws roughly twenty extra samples per response costs orders of magnitude more time than a classifier that reads activations the model has already produced. Latencies are indicative figures from the internal-state detection literature, plotted on a logarithmic scale.

Figure 5.6 illustrates the gap that motivates this entire line of work: a sampling-based check can cost hundreds of times more per response than an internal-state probe.

The internal-state family began with the observation that a model’s hidden layers seem to “know” when the model is producing a falsehood, even when its surface output gives no hint [4]. Building on this, the MIND framework showed that one can generate the training labels automatically, rather than by hand, and still train an effective and fast detector [5]; here MIND refers to the model internal-state-based detection method of Su et al., which we adopt as our starting point. Later work pushed the idea further, either by removing the need for any labels at all [6] or by fusing signals from every layer of the network [1]. These methods are promising, but they have been reported under inconsistent conditions, and crucially most of them were demonstrated on large models running on research-grade hardware. Whether the same signals survive on the smaller open models that a student or a small team can actually run, and how the many proposed features compare on equal footing, is the gap this dissertation sets out to fill.

1.2 Problem Statement

The central question of this work can be stated plainly:

Given an open-weights language model running on free, memory-limited hardware, how reliably can we detect hallucinations by reading internal representations from a single forward pass, and which internal-state signals are worth their cost?

This question has three parts that are easy to conflate and that we deliberately keep separate throughout the dissertation. The first part is the *detection signal*: which quantities, computed from the hidden states and the output distribution, carry information about whether the current

generation is faithful. The second part is the *evaluation regime*: a detector trained on one kind of text (here, automatically labelled encyclopedia continuations) may work well on similar text but transfer poorly to question answering, summarisation, or dialogue. Reporting a single headline number hides this distinction, so we always separate held-out in-distribution performance from cross-task transfer. The third part is the *compute budget*: a method that needs multiple samples, a singular-value decomposition, or a large auxiliary model may be accurate but impractical on the hardware we target. A useful detector has to be honest on all three axes at once.

A complication specific to this work is that a closely related study exists at the same institute, conducted independently in a different research group, which targets the same benchmark suite. To keep the present contribution clearly its own, the design of our detector deliberately differs from that prior study at the level of features and classifier; this separation is explained where the relevant method is introduced in Chapter 3 and respected throughout.

1.3 Research Objectives

The objectives of the dissertation are the following.

1. To re-implement the label-free, internal-state detection pipeline of the MIND framework [5] so that it runs end to end on free cloud GPUs, and to verify its correctness with a small diagnostic model before scaling up.
2. To design three inexpensive internal-state features — layer-wise representation drift, cross-layer variance, and predictive entropy — that are computed from the same forward pass as the existing canonical feature, and to define them in clean, self-contained notation.
3. To assemble a single ablation that compares the canonical feature, the three new features, a set of nine features drawn from the recent literature, and a further set of exploratory features, under one fixed classifier and one fixed evaluation protocol.
4. To evaluate this ablation against four established baselines and two simple reference methods across a multi-task benchmark suite spanning question answering, summarisation, and dialogue, on several open models of different sizes.
5. To report the outcome honestly: to state which numbers are backed by saved measurements, to keep in-distribution and transfer results apart, and to avoid any claim of superiority that the measurements do not support.

1.4 Thesis Contributions

The contributions of this dissertation are modest by design and are stated as such.

A free-tier-reproducible comparison. The primary contribution is a unified, reproducible comparison of internal-state hallucination features across multiple open LLMs, built to run

within the memory and time limits of free cloud notebooks. Every configuration shares the same automatically generated training data, the same classifier, the same train/test split, and the same evaluation rows, so that differences in measured performance can be attributed to the features rather than to incidental differences in setup.

Three forward-pass features and a clean ablation. We introduce three additional internal-state signals on top of the canonical hidden state and place them in an ablation alongside literature features and exploratory features. The ablation isolates what each family of features adds, rather than reporting only a single combined system.

An honest, regime-separated account of the results. We show that, on the model for which we have a complete set of measurements, the richer feature configurations are competitive with the strongest baseline but do not surpass it, and that a single benchmark dominates the averages. We further show that cross-task transfer is markedly harder than in-distribution detection. We treat these findings, including the ones that are unflattering to the proposed features, as the result.

It is worth stating clearly what this dissertation does *not* claim. It does not claim a new state-of-the-art detector, it does not claim to beat any prior method on its own benchmark, and it does not present results for experiments whose measurement files are not yet complete. Where an experiment is still running, it is marked as pending.

1.5 Thesis Organization

The remainder of the dissertation is organised as follows.

Chapter 2 provides the background needed to read the rest of the work: how transformer language models produce hidden states, what a hallucination is in precise terms, and how detection is measured.

Chapter 3 surveys the literature on hallucination detection, organising it into four families and positioning the present work within the internal-state family. It is here that the related institute study is cited once, as adjacent prior work, with an explicit account of how our method differs.

Chapter 4 presents the methodology: the automatic label generation, the canonical feature and the three new internal-state features in full notation, the wider feature families, the classifier, the baselines, and the free-hardware implementation.

Chapter 5 reports the experiments. It presents the verified results, separates in-distribution from transfer performance, discusses the strengths and weaknesses of the proposed features, and marks the experiments that are still in progress.

Chapter 6 summarises what was found, states the limitations without softening them, and lays out concrete next steps.

Chapter 2

Background

This chapter sets out the minimum background needed to follow the rest of the dissertation. It explains how a transformer language model turns text into internal vectors, states precisely what we mean by a hallucination, frames detection as a classification problem, and defines the metrics used to score a detector. A reader already familiar with transformer internals and detection metrics can skim this chapter and move on to Chapter 3.

2.1 Transformer language models and their hidden states

Modern language models are built from the transformer architecture [7]. The input text is first broken into *tokens* — sub-word units — and each token is mapped to a vector. These vectors then pass through a stack of identical layers. Each layer mixes information between token positions using a mechanism called *self-attention*, and then transforms each position independently through a small feed-forward network. The output of every layer is, again, one vector per token. We write $\mathbf{z}_\ell^{(t)}$ for the vector that layer ℓ produces at token position t , and we call it the *activation* or *hidden state* at that layer and position. A model with L layers therefore produces a whole stack of activations $\mathbf{z}_1^{(t)}, \mathbf{z}_2^{(t)}, \dots, \mathbf{z}_L^{(t)}$ for each position it processes.

After the final layer, the model multiplies the last activation by an output matrix to obtain a score for every possible next token, and a softmax turns these scores into a probability distribution $p_t(\cdot)$ over the vocabulary. The token with the highest probability is usually emitted, the text grows by one token, and the process repeats. Two facts about this picture matter for the rest of the dissertation. First, the entire stack of activations is computed anyway during normal generation, so anything we derive from it is essentially free. Second, the activations are high-dimensional — a few thousand numbers per token for the models we study — and they are believed to encode not only *what* the model is saying but also something about *how sure* and *how grounded* it is. The central bet of internal-state detection is that this “something” can be read out.

The dimensionality of a model’s hidden state, written d , and its number of layers L vary with model size. The models used in this dissertation range from a small diagnostic model with a

few hundred million parameters up to seven-billion-parameter models; the six-billion-parameter model that anchors our results has $d = 4096$ and $L = 28$ layers. These sizes are reported in full in Chapter 4.

2.2 What counts as a hallucination

The word “hallucination” covers more than one phenomenon, and being careful about the distinctions avoids confusion later. Following the standard survey [2], we separate two axes.

The first axis is *intrinsic* versus *extrinsic*. An intrinsic hallucination contradicts the material the model was given: a summary that reverses what an article actually said is intrinsic. An extrinsic hallucination adds content that cannot be checked against the given material — it may happen to be true or false in the world, but it is unsupported by the input.

The second axis is *faithfulness* versus *factuality*. Faithfulness asks whether the output is consistent with the prompt or source document; factuality asks whether it is consistent with the real world. A model can be faithful but not factual (it accurately reflects a flawed source) or factual but not faithful (it states a true fact the source never mentioned). Different benchmarks target different corners of this space, which is one reason a detector that does well on one benchmark may do poorly on another.

A third consideration is *granularity*: a hallucination label can be attached to a whole passage, to a sentence, or to an individual fact. Finer labels are more useful but harder to obtain. The approach we build on produces labels automatically at the level of a generated continuation, which sidesteps the cost of human annotation; the price is that the labels are only as good as the automatic procedure that creates them, a point we return to in Chapter 4.

Throughout the dissertation we use the operational definition adopted by the method we extend: a hallucination is a coherent but factually incorrect span of generated text, where “incorrect” is judged against an external reference such as an encyclopedia article or a gold question–answer pair.

2.3 Detection as a classification problem

Once labels exist, detection becomes a supervised classification problem. For each generated example we form a feature vector $\mathbf{x}$ from the model’s internal state, and we attach a binary label y , where $y = 0$ marks a faithful generation and $y = 1$ marks a hallucinated one. A classifier is then trained to estimate $P(y = 1 \mid \mathbf{x})$. In this work the classifier is a multi-layer perceptron, abbreviated MLP: a feed-forward neural network of several layers that maps the feature vector to a probability. The detector’s quality depends almost entirely on whether $\mathbf{x}$ carries the right information, which is why the design of the features — the subject of Chapter 4 — is where the real work lies.

A subtle but important point is the difference between two evaluation regimes. If the classifier

is trained and tested on the same kind of text, we measure *in-distribution* detection. If it is trained on one kind of text (encyclopedia continuations) and tested on another (question answering, summarisation, dialogue), we measure *transfer*. Transfer is the harder and more realistic setting, and as Chapter 5 shows, the two regimes can give very different numbers for the same detector. We never average across them.

2.4 How detection is measured

Because a detector outputs a probability, it can be scored either as a ranking or as a calibrated estimate. We use the following metrics, each defined here at first use.

AUROC. The area under the receiver-operating-characteristic curve, abbreviated AUROC, measures how well the detector *ranks* hallucinated examples above faithful ones. It equals the probability that a randomly chosen hallucinated example receives a higher score than a randomly chosen faithful one. A perfect detector scores 1.0, and a detector that guesses scores 0.5. AUROC is the headline metric in most of the internal-state literature because it does not depend on a particular decision threshold, and it is the metric we use for cross-method comparison.

Accuracy, precision, recall, F1. These threshold-dependent metrics summarise the confusion matrix once a cut-off is chosen. Accuracy is the fraction of correct decisions; precision is the fraction of flagged examples that really were hallucinated; recall is the fraction of true hallucinations that were flagged; and F1 is the harmonic mean of precision and recall. They are useful but can be misleading when the two classes are imbalanced or when the classifier collapses to predicting a single class, a failure mode we observe at small data scales.

Brier score and ECE. A detector is well *calibrated* if a stated probability of, say, 0.8 is right about 80% of the time. The Brier score is the mean squared difference between the predicted probability and the true label; the expected calibration error, abbreviated ECE, bins predictions by confidence and measures how far the average confidence in each bin is from the observed accuracy. Lower is better for both. We report these where our measurement files contain them, because a calibrated probability is more useful in practice than a bare yes/no verdict.

A final caution applies to all of these numbers. When a benchmark is small, a metric estimated on it is noisy, and a single easy or hard dataset can dominate an average computed over several benchmarks. We flag both effects explicitly when they occur in Chapter 5, rather than letting a headline average paper over them.

Chapter 3

Related Work

This chapter reviews the literature on hallucination detection and locates the present dissertation within it. We group the field into four families, sketched in Figure 3.1, and devote the most attention to the internal-state family that this work extends. The chapter closes by describing the benchmarks the area uses, the open problems it has identified, and the specific slot this dissertation occupies.

3.1 Why surface-overlap metrics are not enough

The earliest way to judge a generation was to compare it, word for word, against a reference using overlap scores such as ROUGE or BLEU. These remain common as headline numbers, but they measure surface similarity, not truth. The standard cautionary example makes the point: the sentence “Paris is the capital of Germany” shares almost every word with “Paris is the capital of France” and would earn a near-perfect overlap score, despite being false. Surface form is a poor proxy for meaning, and a worse proxy for factual correctness. The field has therefore moved to *semantic* detectors that do not rely on word overlap, and these fall into the four families below.

3.2 Family I: self-evaluation and calibration

The first idea is to ask the model to judge itself. The line begins with work showing that large models are often reasonably *calibrated*: the probability they assign to an answer tracks how often that answer is correct [8]. Two probes from that work are still influential. The first reads the probability the model assigns to the token “True” after being asked to evaluate its own answer; the second trains a small head to predict, from the question alone, whether the model is likely to answer correctly. Both establish that a usable uncertainty signal exists inside the model, and both are ancestors of the probing classifiers discussed later. Their limitation is that self-evaluation tends to generalise only partially: a probe tuned on one task degrades under distribution shift, a recurring theme that returns in our own transfer results.

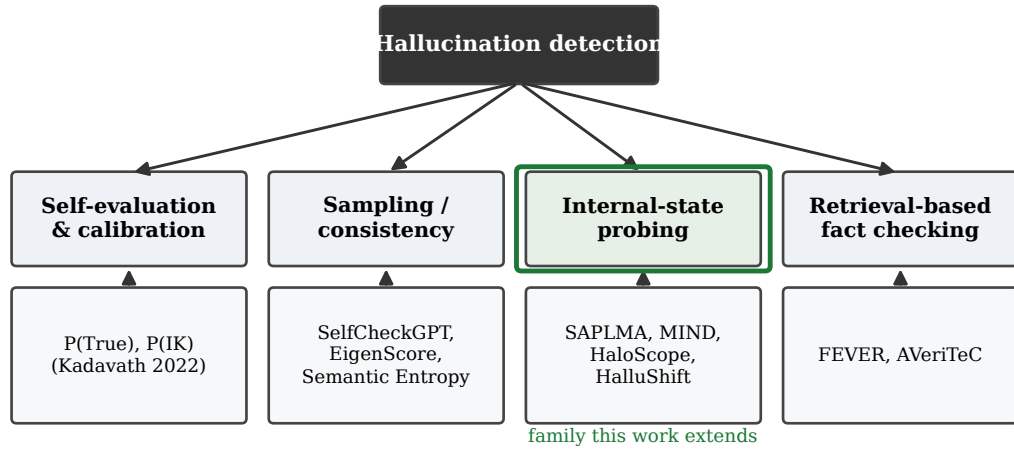


Figure 3.1: Four families of hallucination detection. This dissertation works in the internal-state probing family (outlined), which reads a model’s own activations and is the cheapest to run at inference time.

3.3 Family II: sampling and consistency

A second family ignores the internals entirely and treats the model as a black box. The idea is simple: generate an answer, then draw several more samples at a higher temperature, and check whether they agree. Factual content tends to stay stable across samples, while fabricated content tends to wobble. The canonical method, SelfCheckGPT [3], implements this consistency check at several levels of abstraction, from word-overlap to a natural-language-inference model (abbreviated NLI: a classifier that decides whether one sentence entails or contradicts another). On a benchmark of generated biographies it detects non-factual sentences well and needs no labels, no internals, and no retrieval — only the ability to sample.

The weakness of this family is cost. Drawing roughly twenty extra generations per answer makes detection many times more expensive than the original generation, which is exactly the gap that motivates internal-state methods. Related sampling-based methods include semantic entropy, which clusters samples that mean the same thing and measures the entropy over those clusters [9], and an embedding-space method, EigenScore, which inspects the eigenvalues of the covariance of several sampled generations to gauge how semantically spread out they are [10]; here EigenScore is the scoring rule from the INSIDE method, which we later use as a baseline. These methods are accurate but inherit the sampling cost, and semantic entropy additionally needs an auxiliary NLI model loaded alongside the generator.

3.4 Family III: internal-state probing

The third family — the one this dissertation extends — asks whether truth is written into the activations themselves. If it is, a detector can read it from a single forward pass, with no sampling and no retrieval.

3.4.1 SAPLMA: hidden states reveal untruths

The family is usually traced to the finding that a model’s hidden states “know” when the model is producing a falsehood, even when the surface output gives no sign [4]. That work, SAPLMA, hand-built a corpus of true and false statements, fed each statement to the model, and trained a small feed-forward classifier on the activations the model produced while reading it. A mid-network layer carried the strongest signal, and the probe clearly beat surface-probability baselines, confirming that the relevant information lives in the hidden state rather than in the output logits. The two limitations of this work — its labels were produced by hand, which caps the data scale, and its statements were *read* rather than *generated* by the model — set the agenda for what followed.

3.4.2 MIND: automatic labels and real-time detection

The method this dissertation builds on, MIND [5], removes the hand-labelling bottleneck. It generates its own training data from encyclopedia text: it truncates an article just before a named entity, lets the model continue, and labels the continuation automatically by whether the model’s prediction matches the entity that the article actually had. When the model recovers the right entity the continuation is treated as faithful; when it produces a different entity the continuation is, by construction, factually wrong with respect to the article, and the original suffix is spliced back so that the two versions differ only in the entity span. No human annotation is needed, and the procedure can be run afresh for each target model.

For its detection feature, MIND uses the contextualised vector at the *last token of the last layer* — what we will call the *canonical feature* — and feeds it to a small multi-layer perceptron. The authors show this single view is competitive with richer combinations of layers and positions, and, crucially, that the overhead at inference time is a few percent of the generation cost rather than the many-fold cost of sampling methods. They also report a property that shapes our entire protocol: a detector trained on data from one model transfers poorly to a different model, so the training data must be generated by the same model it will be used to monitor. We adopt MIND’s data generation, its canonical feature, and its per-model discipline as our backbone.

3.4.3 HaloScope: labels from a latent subspace

HaloScope [6] offers a different way to obtain labels without human effort. It collects activations from a pool of unlabelled generations, performs a singular value decomposition (abbreviated

SVD: a factorisation that exposes the principal directions of variation), and scores each sample by how strongly it projects onto the leading directions. The claim is that faithful and hallucinated samples separate along these directions even before any labelling, so a threshold on the projection score yields pseudo-labels, which then train a small classifier. HaloScope also reports that the most informative layer is model-dependent, sitting in the middle of the network for some models and near the end for others — an observation directly relevant to the layer-choice question we discuss below. We use HaloScope as one of our baselines.

3.4.4 HalluShift: multi-layer fusion with probability features

The most directly related prior work is HalluShift [1], a study carried out independently in a different research group at the same institute, on the same benchmark suite this dissertation targets. Because of that overlap, we describe it precisely and state exactly how the present method differs; HalluShift also appears later as one of our baselines.

HalluShift generalises the internal-state idea in three ways. First, instead of choosing a single layer it draws signals from *every* layer, computing, between successive layers, both a Wasserstein distance (an optimal-transport distance between the layers’ value distributions) and a cosine similarity, for hidden states and for attention maps; an automatic, range-wise selection step then keeps the informative subset. Second, it augments these distribution-shift signals with three features taken from the model’s output probabilities. Using $P_{\max}$ and $P_{\min}$ for the largest and smallest token probabilities at a generation step, the three are, in their corrected forms: a minimum-confidence feature equal to the smallest per-step maximum probability, $\min_t P_{\max}^{(t)}$; a confidence-spread feature equal to the largest per-step gap, $\max_t (P_{\max}^{(t)} - P_{\min}^{(t)})$; and a confidence-gradient feature equal to the mean absolute change in confidence between adjacent tokens. Third, it fuses everything in a compact two-layer network trained with a metric-learning objective and a single output unit, using soft labels derived from a learned text-similarity metric (BLEURT, a model-based measure of how close a generation is to a gold reference). On the question-answering and dialogue benchmarks it reports strong numbers and stands as a recent high-water mark for internal-state detection on this suite.

How this dissertation differs. Our method shares only the broad premise — read internal state, train a small classifier — and is deliberately different in every concrete design choice, summarised in Table 3.1. Where HalluShift measures distribution shift with a Wasserstein distance between layer distributions, we use the cosine distance between *adjacent* layer vectors together with an L_2 measure of how much the layer vectors vary; these are intra-sample geometric quantities, not optimal-transport statistics. Where HalluShift derives three features from token probabilities, we use the Shannon entropy of the per-step distribution and its mean over the generation — an information-theoretic summary rather than minimum/spread/gradient statistics. Where HalluShift uses a two-layer network with a metric-learning loss and automatic feature selection, we use a five-layer perceptron trained with ordinary cross-entropy and a *fixed* concate-

Table 3.1: Design separation between HalluShift [1] and the detector developed in this dissertation. The two share only the broad premise of internal-state probing.

Design axis	HalluShift (prior study)	This dissertation
Cross-layer signal	Wasserstein distance + cosine similarity between layer distributions; range-wise selection	Cosine distance of adjacent layers + L_2 cross-layer variance; fixed concatenation
Probability signal	Minimum confidence, confidence spread, confidence gradient	Shannon entropy of the per-step distribution and its mean
Classifier	Two-layer network, metric-learning loss, single sigmoid output	Five-layer perceptron, cross-entropy loss, binary softmax output
Feature selection	Automatic, range-wise	None; features are concatenated as specified
Mathematical character	Inter-distribution (optimal transport)	Intra-sample (geometry and information theory)

nation of features, with no selection step. These are not cosmetic differences: they place the two methods on different mathematical footings, and they are held fixed as design commitments for the whole study.

3.4.5 Recent developments

Several 2025 papers refine the internal-state line and inform our feature design. Decoding-by-contrasting-layers work shows that factual knowledge is concentrated in particular layers and that contrasting the output projections of late and early layers sharpens factuality [11]; the same projection idea, applied as a read-out rather than a decoder, underlies one of our literature features. Attention-only work demonstrates that the ratio of attention paid to the context versus to freshly generated tokens is by itself a strong signal for contextual hallucination [12]. Token-selection work frames detection as choosing the few tokens whose representations are most telling, rather than pooling over all of them [13]. And reasoning-aware work decomposes the internal signal into a within-layer spread and an across-layer drift, both of which we echo in simplified form [14]. We draw individual scalar features from these studies in Chapter 4, while keeping our own three core features distinct from all of them.

3.5 Family IV: retrieval-based fact verification

A fourth family treats detection as fact-checking against an external knowledge base: parse the generation into claims, retrieve evidence, and decide whether the evidence supports or refutes

Table 3.2: Benchmark coverage of representative prior methods. A check mark indicates the method reports results on that benchmark in its own paper. This dissertation evaluates all configurations on the full set.

Benchmark	MIND	SelfCheckGPT	SAPLMA	HaloScope	Kadavath	HalluShift
TruthfulQA	–	–	–	✓	✓	✓
TriviaQA	–	–	–	✓	✓	✓
CoQA	–	–	–	✓	–	✓
TyDi QA (en)	–	–	–	✓	–	✓
HaluEval-QA	–	–	–	–	–	✓
HaluEval-Summ	–	–	–	–	–	✓
HaluEval-Dialog	–	–	–	–	–	✓

each claim. The reference benchmark for this line is FEVER [15], and a related strand pushes evaluation to the level of individual atomic facts [16]. This family is powerful and works even on closed models accessed only through an interface, but it is fundamentally slower, depends on retrieval quality, and needs an external corpus. It is therefore complementary to, rather than competitive with, the internal-state methods we study: the two could be combined in a production system, but they answer different questions.

3.6 Benchmarks and metrics used in this area

The benchmarks that recur in this literature, and that we use in Chapter 5, span several task types. For open-domain question answering the common choices are TruthfulQA [17], which targets questions that provoke common misconceptions; TriviaQA [18]; the conversational benchmark CoQA [19]; and the English portion of TyDi QA [20]. For directly labelled hallucination examples, HaluEval [21] provides question-answering, summarisation, and dialogue subsets in which a model-written answer has been programmatically altered to introduce a fabricated span. Three further question-answering sets broaden the coverage: Natural Questions in its open form [22], the multi-hop set HotpotQA [23], and the entity-centric PopQA [24]. Across this literature, AUROC is the dominant comparison metric because it is threshold-free, with accuracy and F1 reported alongside it.

The coverage of these benchmarks is uneven across prior methods, as Table 3.2 shows. Only a few internal-state methods report on the full question-answering suite, and the directly labelled HaluEval subsets are reported by even fewer. Part of the contribution of this dissertation is to evaluate every configuration on a single, consistent ten-benchmark sweep.

3.7 Open problems

The literature converges on a handful of unresolved issues that bear directly on this work. *Cross-model training data* appears unavoidable: probes do not transfer well between models, so each

target model needs its own training corpus. *Layer selection* is unsettled, with different methods preferring middle or final layers on different models and no rule that holds everywhere. *Transfer across tasks* is under-studied: most methods are tuned and tested on similar text, and it is unclear how much a probe trained on encyclopedia continuations carries over to question answering or dialogue. *Calibration* is usually neglected in favour of ranking metrics, even though a calibrated probability is more useful downstream. And the entire line presumes *white-box access* to hidden states, which restricts it to open-weights models — increasingly the deployment reality for applications where these very signals are needed.

3.8 Positioning of this dissertation

Against this backdrop, the present work occupies a deliberately narrow and honest slot. It does not propose a new architecture; it adopts MIND’s framework and asks a measurement question. Concretely, it makes three moves. It *re-implements* the internal-state pipeline so that it runs on free hardware and verifies correctness on a tiny model before scaling. It *extends* the canonical feature with three inexpensive forward-pass signals and places them in a controlled ablation alongside literature features and exploratory features. And it *evaluates* every configuration, together with four established baselines and two simple reference methods, on one consistent multi-benchmark sweep across several open models of different sizes — from a one-billion-parameter model up to seven-billion-parameter models. Two methods that were considered as baselines, semantic entropy [9] and the attention-only Lookback Lens [12], were ultimately left out of the final baseline set for reasons of running time and paradigm overlap, as explained in Chapter 4; we acknowledge them here as relevant prior art. The result is not a victory over any single method but a clearer, reproducible picture of which internal-state signals are worth their cost, and where they break down.

Chapter 4

Methodology

This chapter describes the detector in full. It begins with the overall pipeline, then covers the automatic generation of training labels, the canonical feature and the three new internal-state features in self-contained notation, the wider feature families used in the ablation, the classifier, the evaluation protocol, the baselines, and finally the models and the free-hardware setup under which everything runs.

4.1 Overall pipeline

At a high level the detector reads signals from one forward pass of a frozen language model and feeds them to a small trained classifier; the language model’s own weights are never touched. Figure 4.1 shows the flow for a single input. What makes this practical is that everything the detector needs — the final hidden state, the layer-by-layer activations, the attention maps, and the output distribution — is already produced when the model reads a prompt and writes a short continuation. Reading those signals off therefore costs almost nothing on top of ordinary generation.

The full study runs as five stages, each treated in its own section below.

Stage 1. Data generation turns raw encyclopedia text into labelled training examples with no human annotation, and does so separately for each target model (Section 4.2).

Stage 2. Feature extraction runs the model once over each example and records the canonical hidden state together with the scalar signals the configurations draw on (Sections 4.4–4.5).

Stage 3. Classifier training fits one small perceptron per configuration on a fixed split of the generated data (Section 4.7).

Stage 4. In-distribution evaluation scores each trained detector on held-out text of the same kind it was trained on.

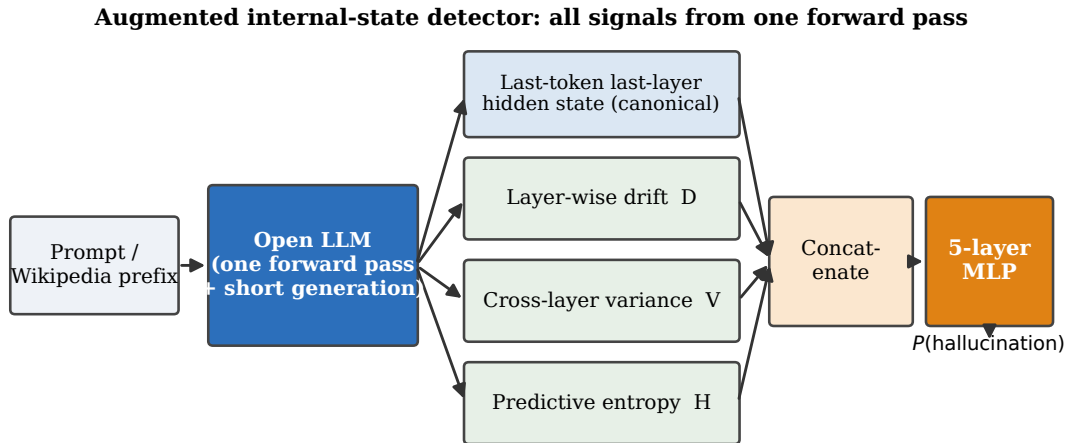


Figure 4.1: The augmented internal-state detector. Every signal is read from one forward pass of a frozen model, so detection adds negligible cost over normal generation.

Stage 5. Transfer evaluation scores each detector on ten external benchmarks it never saw during training, alongside six baseline detectors run through the same pipeline on the same rows (Sections 4.8 and 4.9).

Holding stages one, three, four and five fixed and changing only the feature set in stage two is exactly what turns the study into a controlled comparison of *features* rather than of incidental differences in setup.

4.2 Automatic label generation

Supervised detection needs labelled examples, and labelling generations by hand does not scale. We adopt the label-free procedure of the MIND framework [5], illustrated in Figure 4.2 and stated as Algorithm 1. We take an article, keep its first sentences, and find a named entity that occurs after the first sentence. We truncate the text just before that entity and let the model predict the continuation. If the entity the article actually had is among the model’s top few predicted first tokens, the model has stayed faithful, and we label the continuation $y = 0$. If the model instead produces a different entity, the continuation is factually wrong with respect to the article; we let it generate on, then splice the article’s original suffix back so that the faithful and hallucinated versions differ only in the entity span, and we label it $y = 1$. The threshold for “among the top few” is the top four predictions throughout.

Two properties of this procedure matter. It produces balanced, per-model data at no annotation cost, and — because probes do not transfer well between models — it is run *separately for each model* we study, so the training data always comes from the same model the detector will later

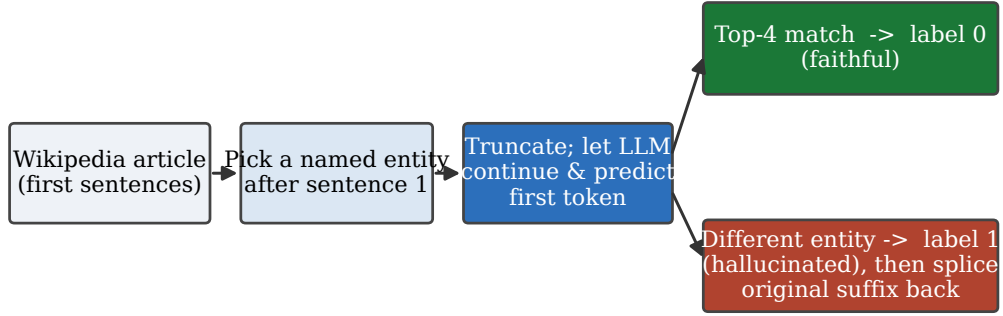
Automatic (label-free) data generation, per target model

Figure 4.2: Automatic, label-free data generation. The model’s own first-token prediction decides the label; no human annotation is used, and the procedure is re-run separately for each target model.

Input: article text, target model M , top- k threshold $k = 4$

- 1: keep the first sentences; find a named entity e occurring after sentence one
 - 2: truncate the text just before e to form a prefix x
 - 3: let M predict the next-token distribution given x
 - 4: **if** e ’s first token is among M ’s top- k predictions **then**
 - 5: label the continuation faithful ($y \leftarrow 0$)
 - 6: **else**
 - 7: let M generate until the original post-entity anchor reappears
 - 8: splice the article’s original suffix back onto the generated span
 - 9: label the continuation hallucinated ($y \leftarrow 1$)
 - 10: **end if**
 - 11: **return** labelled example ($x \parallel$ continuation, y)
-

Algorithm 1: Automatic label generation for one article

monitor. For the larger models we generate one thousand examples per class, two thousand in total.

A few implementation details make the procedure concrete. The raw text is streamed from English Wikipedia, so no fixed corpus has to be downloaded in advance. For each article we keep the opening passage, run a named-entity recogniser over it, and pick at random one entity that begins after the first sentence. We tokenise the article up to that entity and let the model generate a short window of tokens. The top-four rule is applied twice: once on the entity’s first token, which decides the label, and once step by step, to find where the model’s continuation rejoins the original article, so that the article’s true suffix can be grafted back on cleanly and the faithful and hallucinated versions differ only in the entity span. The per-step output distributions produced during this generation are stored at the same time, which is what later makes the predictive-entropy feature available without a second forward pass. Generations that yield an

empty entity, that merely echo the original entity, or that never rejoin the article are discarded, and the surviving examples are kept balanced between the two classes.

4.3 The canonical feature

The backbone feature, which we call the *canonical feature*, is the contextualised vector at the last token of the final transformer layer [5]. Writing $\mathbf{z}_\ell$ for the last-token activation at layer ℓ and L for the number of layers, the canonical feature is simply $\mathbf{h} = \mathbf{z}_L$. Its dimension equals the model’s hidden width d — for the six-billion-parameter model that anchors our results, $d = 4096$. Every configuration in the ablation includes this feature; the question the ablation answers is what, if anything, the additional scalars buy on top of it.

4.4 Three internal-state features

The core proposal of this dissertation is to summarise the model’s internal behaviour during a generation with three inexpensive scalars, all computed from activations that the forward pass already produced. We use fresh notation throughout: $\mathbf{z}_\ell$ is the last-token activation vector at layer ℓ , and $p_t(\cdot)$ is the model’s token distribution at generation step t .

Layer-wise representation drift. As information moves up the layer stack, the representation of a token changes. We measure how much by the cosine distance between adjacent layers,

$$d_\ell = 1 - \frac{\mathbf{z}_\ell \cdot \mathbf{z}_{\ell+1}}{\|\mathbf{z}_\ell\| \|\mathbf{z}_{\ell+1}\|}, \quad \ell = 1, \dots, L-1, \quad (4.1)$$

and summarise the whole trajectory by its mean,

$$D_{\text{mean}} = \frac{1}{L-1} \sum_{\ell=1}^{L-1} d_\ell. \quad (4.2)$$

A faithful continuation tends to settle into a stable trajectory; large or erratic drift is a candidate signal of trouble.

Cross-layer variance. A complementary view asks how spread out the layer representations are around their own average. With the layer-wise mean $\bar{\mathbf{z}} = \frac{1}{L} \sum_{\ell=1}^L \mathbf{z}_\ell$, we define

$$V_{\text{last}} = \frac{1}{L} \sum_{\ell=1}^L \|\mathbf{z}_\ell - \bar{\mathbf{z}}\|_2^2. \quad (4.3)$$

Whereas drift is a sequential, layer-to-layer quantity, the variance is a global measure of dispersion across the whole stack. Only the scalar V_{last} is stored; the mean $\bar{\mathbf{z}}$ is internal to its computation.

Predictive entropy. The third scalar looks at the output distribution rather than the hidden states. At each generation step the Shannon entropy of the token distribution is

$$H^{(t)} = - \sum_{w \in V} p_t(w) \log p_t(w), \quad (4.4)$$

where V is the vocabulary, and we average it over the T generated steps,

$$H_{\text{mean}} = \frac{1}{T} \sum_{t=1}^T H^{(t)}. \quad (4.5)$$

High predictive entropy means the model was unsure as it generated, which is weak but useful evidence of hallucination.

The three scalars $\{D_{\text{mean}}, V_{\text{last}}, H_{\text{mean}}\}$ are concatenated onto the canonical feature. Crucially, all three are intra-sample summaries from a single pass: they involve no extra sampling, no auxiliary model, and no optimal-transport computation, which keeps them clearly distinct from the prior study discussed in Section 3.4 and keeps their cost negligible.

4.5 Two further feature families

To place the three core features in context, the ablation also evaluates two further families of scalar features, every one of them read from the same single forward pass. The first family gathers signals proposed in recent internal-state work; the second is a more exploratory set of geometric and attention-based signals assembled for this study.

4.5.1 Literature features

Nine scalars are taken from the recent literature. Each is computed from the hidden states, the attention maps, or the output distribution of the single pass, and each is reduced to one number per example.

- (1) **Lookback ratio** — the share of the final-layer attention, at the last position, that falls on the prompt tokens rather than on the freshly generated ones; a low value means the model has stopped attending to its context [12].
- (2) **Attention-sink score** — the average fraction of attention mass that collapses onto a small set of “sink” tokens (sentence punctuation and special tokens), averaged over layers [25].
- (3) **Spectral dispersion** — the log-determinant of the covariance of the last-layer token representations, a lightweight version of the eigenvalue score used by the INSIDE method [10].
- (4) **Internal-update ratio** — the mean, over layers, of the size of each layer’s update to the last-token representation relative to that representation’s own norm; it measures how much work the upper layers are still doing.

- (5) **Logit-lens divergence** — the Jensen–Shannon divergence (abbreviated JSD: a symmetric measure of how different two probability distributions are) between the vocabulary distribution read off from a middle layer and the final output distribution, following the contrast-between-layers idea [11].
- (6) **Attention-head entropy** — the average entropy of the last-position attention distribution, taken across heads and layers; diffuse attention gives a high value.
- (7) **Confidence margin** — the mean gap between the largest and second-largest next-token probabilities over the generated positions, in the spirit of adaptive token-selection work [13].
- (8) **Token rank** — the mean rank of the token the model actually emitted within its own predicted distribution at each step.
- (9) **Intra-layer dispersion** — the mean distance of the last-layer token embeddings from their centroid, a within-layer spread measure in the spirit of reasoning-aware detection [14].

A tenth feature in this family, a supervised mid-layer probe of the SAPLMA kind, is deferred because it would require training a separate probe inside the pipeline; we record its omission rather than dropping it silently. The nine features above are exactly those used in the configurations that follow.

4.5.2 Additional exploratory features

A second set of six signals probes less conventional geometry and attention behaviour. They were added after the literature set, and they are not a throwaway: as Chapter 5 reports, one combination of them gives the strongest result on one of the two fully-measured models, so we treat them as part of the contribution rather than an afterthought.

- (1) **Unembedding-residual projection** — an eight-number projection of the last-token hidden state onto the directions that the model’s output (unembedding) matrix uses *least*, i.e. energy the model carries internally but barely turns into output. This is the only feature in the study that contributes more than one scalar (eight).
- (2) **Layer-trajectory curvature** — how sharply the path traced by the representation bends from one layer to the next, summarised by its mean and its maximum (two numbers).
- (3) **Confidence trajectory** — the slope and the variance of the model’s log-confidence across the generated tokens (two numbers); a falling or jittery confidence trajectory is a candidate warning sign.
- (4) **Effective attention rank** — the effective number of directions a middle-layer attention map spreads over, read from the entropy of its singular values.

Configurations = canonical backbone + chosen feature families (fixed concatenation)

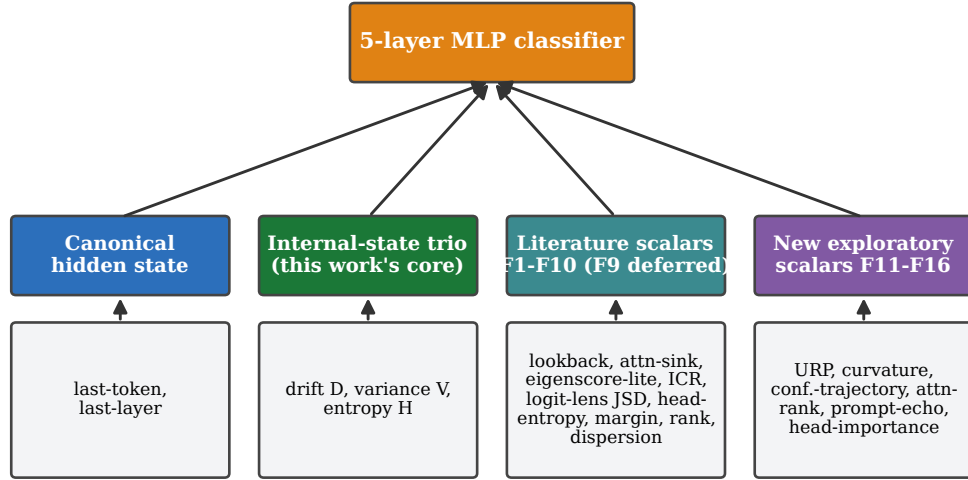


Figure 4.3: The feature families that make up the configurations. Each configuration is the canonical backbone plus a fixed concatenation of chosen scalar families; there is no automatic feature selection.

- (5) **Prompt-echo alignment** — the cosine similarity between the average middle-layer representation of the prompt tokens and that of the generated tokens; it measures how much the generation still echoes the prompt.
- (6) **Head-importance divergence** — how unevenly attention concentration is spread across the heads of a middle layer, expressed as a coefficient of variation.

Used together these six contribute fourteen numbers (the eight-dimensional projection plus six further scalars). Like the core three, every one is an intra-sample quantity from the single forward pass; none requires extra sampling, an auxiliary model, or an optimal-transport computation.

4.6 Feature configurations

The ablation evaluates sixteen configurations, each being the canonical feature plus a chosen subset of scalar features, as laid out in Figure 4.3 and Table 4.1. The configurations move from the canonical feature alone, through each new scalar added singly, to the three-scalar internal-state set, to the literature set, to the exploratory set, and finally to combinations of these. Throughout the dissertation we refer to configurations by a neutral index and a short description; we do not use the working code names from the implementation. The configuration consisting of the canonical feature plus the three internal-state scalars (configuration 5) is the headline proposal, and we call it the *augmented internal-state detector*.

Table 4.1: The sixteen feature configurations. The input dimension is shown for the six-billion-parameter model (hidden width $d = 4096$); for other models the canonical part changes with d while the scalar counts stay the same.

Cfg.	Feature content	# scalars	MLP input dim
C1	Canonical feature only (backbone)	0	4096
C2	Canonical + drift	1	4097
C3	Canonical + cross-layer variance	1	4097
C4	Canonical + predictive entropy	1	4097
C5	Canonical + internal-state trio (<i>this work</i>)	3	4099
C6	Canonical + lookback ratio	1	4097
C7	Canonical + logit-lens divergence	1	4097
C8	Canonical + confidence margin	1	4097
C9	Canonical + lookback/lens/margin trio	3	4099
C10	Canonical + all nine literature features	9	4105
C11	Canonical + internal-state trio + lookback/lens/margin	6	4102
C12	Canonical + internal-state trio + all literature	12	4108
C13	Canonical + unembedding projection + curvature	10	4106
C14	Canonical + confidence trajectory + attn. rank + prompt echo	4	4100
C15	Canonical + unembedding proj. + curvature + confidence	12	4108
C16	Canonical + internal-state trio + all six exploratory	18	4114

4.7 The classifier

Every configuration uses the same classifier, so that differences in measured performance come from the features and not from the model on top of them. The classifier is a five-layer perceptron. The input layer maps the feature vector to 512 units; three hidden layers of 256, 128 and 64 units follow, each with a rectified-linear activation and a dropout of 0.2 after the first; and a final linear layer produces a two-way softmax over {faithful, hallucinated}. It is trained with the cross-entropy loss using the Adam optimiser at a learning rate of 5×10^{-4} and weight decay 10^{-5} , with a batch size of 32, for ten epochs. The scalar and canonical features are standardised by a scaler fitted on the training split only, and the data is split 80/20 into training and test sets with a fixed random seed of 42 so that every configuration sees exactly the same partition. The use of a fixed, five-layer perceptron with cross-entropy and no feature selection is a deliberate design commitment that distinguishes this work from the multi-feature, metric-learning prior study of Section 3.4.

4.8 Evaluation protocol

After training on the encyclopedia-continuation data, each detector is evaluated in two regimes. The *in-distribution* regime uses a held-out split of the same encyclopedia-continuation data.

The *transfer* regime uses a multi-benchmark suite of question-answering, summarisation, and dialogue tasks that the detector never saw during training. Algorithm 2 states the protocol.

To make the comparison fair and cheap, every method — configuration or baseline — is scored on the *same* fixed set of rows from each benchmark: a deterministic first- N slice, with N up to 350 rows per benchmark in the fully unified runs. Because both the configuration runs and the baseline runs read the same rows, their scores are directly comparable. The transfer suite contains up to ten benchmarks: the four question-answering sets TruthfulQA [17], TriviaQA [18], CoQA [19] and TyDi QA [20]; the three directly labelled HaluEval subsets [21]; and three further question-answering sets, Natural Questions [22], HotpotQA [23] and PopQA [24]. Each benchmark is wrapped in a short, fixed instruction prompt appropriate to its task; the prompt wording is held constant across all methods.

Input: trained classifier C , target model M , benchmark suite $\mathcal{D}$, slice size N

- 1: **for** each benchmark $D \in \mathcal{D}$ **do**
- 2: **for** each of the first N rows of D **do**
- 3: run M once on the prompt; collect last-layer hidden state and per-layer activations
- 4: compute the canonical feature and the chosen scalars (drift, variance, entropy, ...)
- 5: standardise with the training scaler; obtain $\hat{p} = C(\mathbf{x})$
- 6: **end for**
- 7: compute AUROC, accuracy, precision, recall, F1 (and calibration where available)
- 8: **end for**
- 9: **return** per-benchmark metrics and their mean

Algorithm 2: Single-pass feature extraction and multi-task evaluation

4.9 Baselines

The detectors are compared against four established methods plus two simple reference methods, all re-implemented inside the same pipeline and scored on the same rows. Table 4.2 summarises them. SAPLMA [4] is a supervised probe on a single intermediate layer’s hidden state. Halo-Scope [6] is the unsupervised spectral method that estimates pseudo-labels from a latent subspace. HalluShift [1] is the multi-feature supervised method of Section 3.4, re-implemented from its description with a thirty-one dimensional feature vector. EigenScore is the sampling-based covariance score from the INSIDE method [10]. To these we add two reference points: MIND, a supervised probe on the final-layer hidden state — effectively the method this work extends [5] — and Perplexity, an unsupervised score equal to the mean token negative log-likelihood (abbreviated NLL), where a higher value of the continuation’s perplexity is taken as weak evidence of hallucination.

Two further methods were considered as baselines and deliberately left out of the final set. Semantic entropy [9] requires drawing several samples per prompt and loading an auxiliary natural-language-inference model, which adds several hours of computation per large model

Table 4.2: Baselines re-implemented inside the unified pipeline. “Dim” is the dimensionality of each method’s feature or score for the six-billion-parameter model.

Baseline	Paradigm	Dim	Signal
SAPLMA	supervised probe	4096	hidden state at a single intermediate layer
HaloScope	unsupervised spectral	4096	best-layer hidden, pseudo-labelled from a subspace
HalluShift	supervised multi-feature	31	layer distribution-shift + attention + probability features
EigenScore	sampling-based (score)	1	log-determinant of the covariance of several samples
MIND	supervised probe	4096	final-layer last-token hidden state
Perplexity	unsupervised (score)	1	mean token negative log-likelihood

and would have broken the free-tier time budget. The attention-only Lookback Lens [12] was dropped because the attention statistics it relies on are already represented among our features and among HalluShift’s, so it added a row without adding a new paradigm. The final baseline set is therefore the four established methods plus the two reference scores.

4.10 Models and free-hardware implementation

A goal of this dissertation is that every experiment runs on free cloud hardware, so that the study can be reproduced without a research budget. The primary platform is a free Kaggle notebook with two T4 GPUs of sixteen gigabytes each, giving thirty-two gigabytes of combined video memory (abbreviated VRAM) through model parallelism, used in the bfloat16 sixteen-bit number format (abbreviated bf16) with no further quantisation. Sessions are capped at nine hours, within a weekly budget of thirty GPU-hours. A free Colab notebook with a single T4 serves as a fallback for the smaller models. The seven-billion-parameter models require both T4 GPUs; a single sixteen-gigabyte car

Chapter 5

Results and Discussion

This chapter presents the experimental results. Two distinctions run through it. First, we keep in-distribution detection — testing on held-out text of the kind the detector was trained on — separate from cross-task transfer, because the two behave very differently and merging them would mislead. Second, we are explicit about which model each number comes from, because the central finding is that the ranking of methods depends on the model. The chapter states which models are complete and which are still in progress, presents the in-distribution results, then the full transfer comparison on the model with the complete sweep, then the second model on which a proposed configuration leads, and closes with the discussion.

5.1 What has been measured

The study spans several models, but they are at different stages of completion, and it would be misleading to present them as a finished sweep. Table 5.1 states the status of each model honestly. One six-billion-parameter model has a complete, fully file-backed set of measurements — all sixteen configurations and all six baselines, on all ten benchmarks, scored on identical rows — and it anchors this chapter. A second model of comparable size has a ten-benchmark comparison of the six baselines against the exploratory-feature configurations, on which a proposed configuration leads; its remaining configurations were only run on an earlier seven-benchmark suite. The remaining large models have generated their training data but do not yet have reported detection results, and are marked accordingly.

5.2 In-distribution detection

We begin with the easier regime, in which the detector is tested on held-out text of the same kind it was trained on. An earlier mid-term evaluation, run on a larger ten-thousand-sample corpus with the canonical feature alone, reported in-distribution AUROC of 0.673 for the three-billion model and 0.592 for the one-billion model, and noted that the larger model carried the stronger

Table 5.1: Status of each model at the time of writing. Only the model with a complete, file-backed set of measurements is used for the headline comparison.

Model	Training examples	Status
GPT-J (6.0 B)	2000	complete (file-backed): 16 configurations + 6 baselines, 10 benchmarks
OPT (6.7 B)	2000	6 baselines + 4 new-feature configs on 10 benchmarks; other 12 configurations
Qwen2.5 (3 B)	2000	in-distribution configurations only (no baseline run)
TinyLlama (1.1 B)	—	earlier in-distribution figure only
Mistral (7 B)	2000	data generated; detection results pending
Falcon (7 B)	2000	data generated; detection results pending
Llama-2 (7 B)	400	gated access; undersized data; results pending
GPT-2 (0.12 B)	1000	smoke test; diagnostic only, not a result

internal signal. These are encouraging numbers, but they are in-distribution and were obtained at a larger data scale than the unified runs reported below.

At the two-thousand-sample scale of the unified pipeline, in-distribution AUROC is more modest. On the three-billion model the configurations sit between roughly 0.39 and 0.47, and on the six-billion model they sit between roughly 0.40 and 0.55. The drop relative to the mid-term figure is consistent with the smaller training set and the harder, fully unified evaluation; it is a reminder that sample size matters and that a single favourable number should not be over-read. Figure 5.1 places the in-distribution and transfer numbers side by side and makes the central point of this chapter visible at a glance: in-distribution detection, while imperfect, is clearly easier than transferring to unseen task types.

5.3 Transfer results on the fully measured model

We now turn to the substantive result: cross-task transfer on the six-billion-parameter model, for which every configuration and baseline was measured on the same rows of all ten benchmarks. Table 5.2 reports the sixteen configurations and Table 5.3 the six baselines, each by mean transfer AUROC; the full per-benchmark matrix is given in Appendix A.

The picture that emerges is mixed, and we report it as such. Ranked by mean transfer AUROC, the strongest method is the supervised mid-layer probe, SAPLMA, at 0.562. The two best feature configurations follow closely: the all-literature configuration at 0.548 and the confidence-trajectory configuration at 0.546, with the configuration that adds the internal-state trio to a small literature subset close behind at 0.525. The simple perplexity reference scores 0.523, ahead of the more elaborate HalluShift, HaloScope, final-layer MIND, and EigenScore baselines, all of which sit between 0.487 and 0.509. Figure 5.2 and Figure 5.3 show these comparisons graphically.

Three observations follow directly from these numbers. First, the three core internal-state

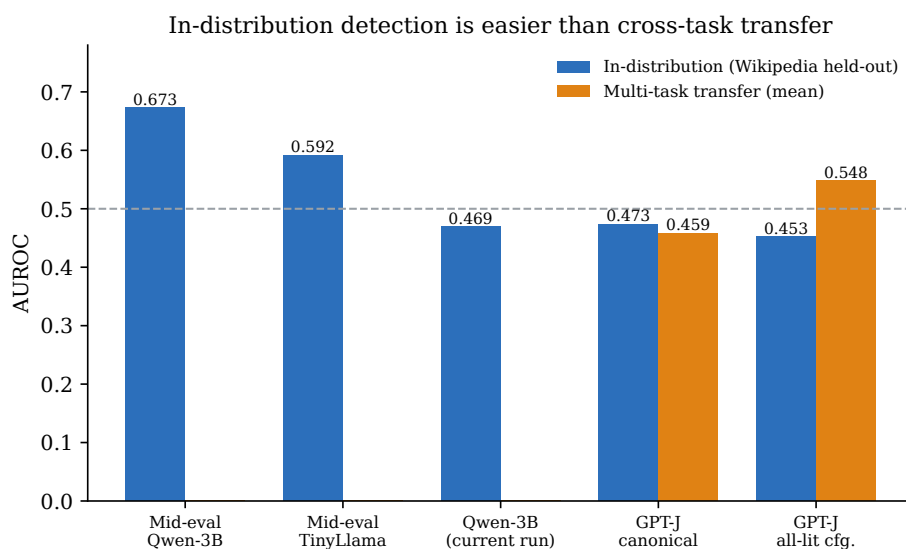


Figure 5.1: In-distribution detection is easier than cross-task transfer. The two left bars are earlier mid-term in-distribution figures at a larger data scale; the rest are from the unified runs. Transfer means are computed over the ten-benchmark suite.

features, taken alone on top of the canonical feature (configuration 5), move the mean transfer AUROC only slightly, from 0.459 to 0.464; the individual additions are mixed, with predictive entropy and the confidence margin helping a little and drift or variance alone not helping. The lift that does appear comes from the richer combinations of many features at once. Second, the strongest baseline reads a *mid-network* layer, while the canonical feature and the MIND baseline read the *final* layer, and the mid-layer probe is clearly better here (0.562 versus 0.487). This echoes the layer-selection problem flagged in the literature: on this model the last layer is not the most informative place to look. Third, none of the methods is far above chance on transfer; the averages sit in the high-0.4 to mid-0.5 range, which is honest evidence that transferring an encyclopedia-trained probe to unseen task types is hard.

5.4 One benchmark dominates the averages

Before reading too much into the rankings, it is important to see how they are formed. Figure 5.4 shows the per-benchmark AUROC for a representative set of methods, and Figure 5.5 traces the same numbers as lines. One benchmark, the question-answering subset of HaluEval, behaves very differently from the rest: the mid-layer probe reaches 0.985 on it, the confidence-trajectory configuration 0.94, and the all-literature configuration 0.84, while on the other nine benchmarks almost every method hovers near 0.5. Because this one column is so high, it pulls the averages up and largely determines the ranking. The honest reading is that the headline averages are carried by a single, unusually separable benchmark, and that on the remaining benchmarks the methods are much closer together and much closer to chance. A ranking that looks decisive on paper is, on inspection, the product of one easy column.

Table 5.2: Six-billion-parameter model: the sixteen feature configurations, by mean transfer AUROC over ten benchmarks, with the in-distribution figure for reference. Higher is better; 0.5 is chance.

Cfg.	Feature content	In-distribution	Mean transfer
C1	Canonical feature only	0.473	0.459
C2	+ drift	0.500	0.447
C3	+ cross-layer variance	0.525	0.445
C4	+ predictive entropy	0.404	0.472
C5	+ internal-state trio (<i>this work</i>)	0.500	0.464
C6	+ lookback ratio	0.524	0.445
C7	+ logit-lens divergence	0.498	0.446
C8	+ confidence margin	0.546	0.476
C9	+ lookback/lens/margin trio	0.481	0.446
C10	+ all nine literature features	0.453	0.548
C11	+ internal-state trio + lookback/lens/margin	0.540	0.525
C12	+ internal-state trio + all literature	0.475	0.468
C13	+ unembedding projection + curvature	0.488	0.451
C14	+ confidence trajectory + attn. rank + echo	0.405	0.546
C15	+ unembedding proj. + curvature + confidence	0.508	0.471
C16	+ internal-state trio + all six exploratory	0.507	0.508

5.5 The second model: a proposed configuration leads

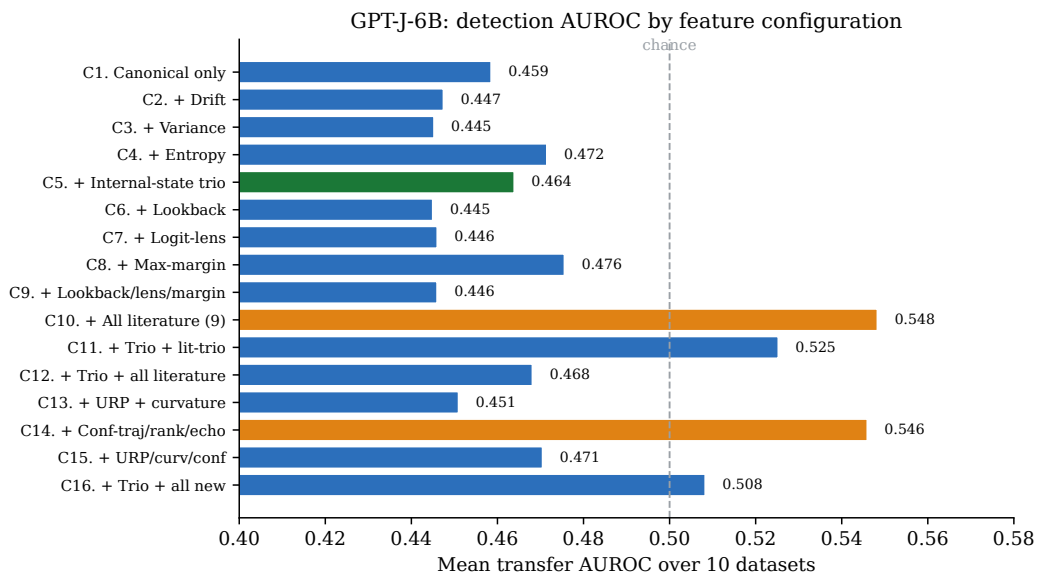
The second six-billion-class model gives a complementary picture that, importantly, points the other way. Here the six baselines and the four configurations built from the additional exploratory features (Section 4.5.2) were evaluated on all ten benchmarks on identical rows; the remaining twelve configurations were only run earlier on the original seven-benchmark suite at a smaller sample size and are not directly comparable, so we set them aside (their averages, near or below 0.44 over seven benchmarks, are diagnostic and appear in Appendix A). Table 5.4 reports the comparable set.

On this model the strongest method overall is a *proposed* configuration: the combination of the confidence trajectory, the effective attention rank, and the prompt-echo alignment reaches a mean transfer AUROC of 0.551, ahead of the simple perplexity reference (0.516), the supervised mid-layer probe (0.514), and the final-layer probe (0.506), with HalluShift, EigenScore, and the spectral baseline further back. The same caution as before applies: the question-answering subset of HaluEval again swings the averages (the leading configuration reaches 0.878 there), so the per-benchmark figures in Appendix A matter more than the mean.

These OPT figures come from the project’s consolidated ten-benchmark comparison. The twelve configurations that use only the literature and core internal-state features were evaluated on the earlier seven-benchmark suite; re-running the unified sweep would place them on the

Table 5.3: Six-billion-parameter model: the six baselines, by mean transfer AUROC over ten benchmarks. The strongest method overall is the supervised mid-layer probe.

Baseline	In-distribution	Mean transfer
SAPLMA (mid-layer probe)	0.419	0.562
Perplexity (mean NLL)	—	0.523
HalluShift	0.519	0.509
HaloScope	0.506	0.508
MIND (final-layer probe)	—	0.487
EigenScore	0.515	0.487

**Figure 5.2:** Mean transfer AUROC of the sixteen configurations on the six-billion model. The three-feature internal-state set (highlighted) gives only a small change over the canonical backbone; the gains come from the richer stacks (highlighted).

same ten-benchmark footing as the four exploratory-feature configurations and the six baselines shown here.

The headline is the contrast with the first model. There, a supervised baseline that reads a middle layer was strongest and the proposed features were competitive but behind; here, a configuration built from the exploratory internal-state features is strongest and clears every baseline. The two models genuinely disagree about which method wins, and that model-dependence — not a single winner — is the finding.

5.6 Models still in progress

Three further seven-billion models have completed the data-generation stage but do not yet have reported detection results, and are listed as pending in Table 5.1. One of them is gated and

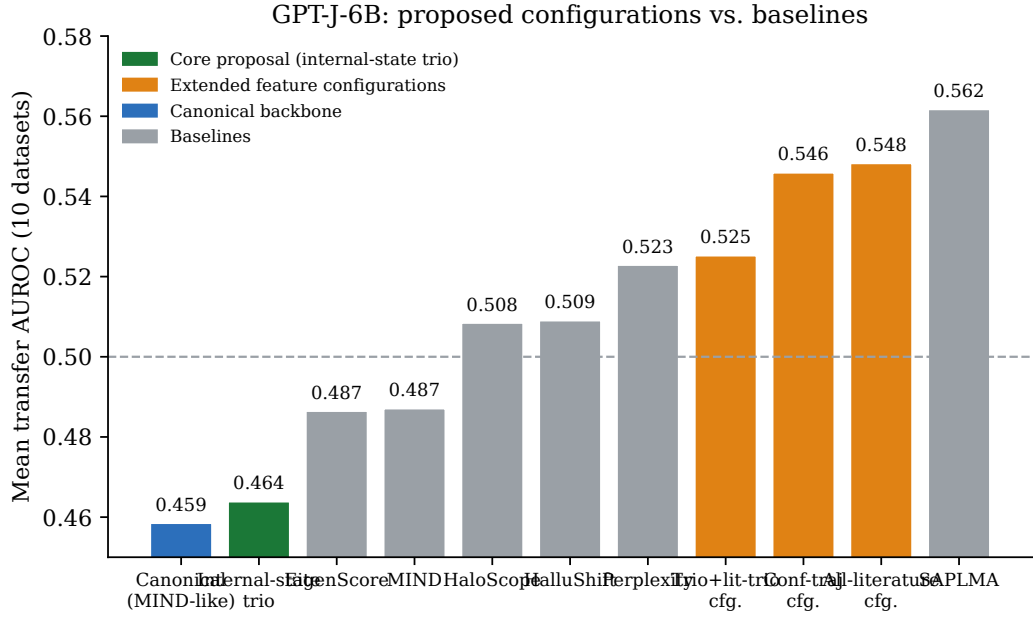


Figure 5.3: The proposed configurations against the baselines on the six-billion model. The best configurations are competitive with the field but do not overtake the strongest baseline, the supervised mid-layer probe.

currently has an undersized training set, which must be regenerated before its numbers will be comparable. The tiny diagnostic model behaves exactly as expected for its scale: its AUROCs scatter around chance and its classifier frequently collapses to a single class on the small test set. We use it only to confirm that the pipeline is correct end to end, and we do not present its numbers as findings.

5.7 Inference cost

Accuracy is only half of what makes a detector usable; the other half is what it costs to run. Every configuration in this study, and four of the six baselines, read their signals from a single forward pass, so they share essentially the same inference cost: the forward pass dominates, and the small classifier that follows adds well under a millisecond. Figure 5.6 makes this concrete. The configurations that use only the hidden state and the literature features sit in the same band as the supervised baselines, around half a second per input on the six-billion model; the configurations that add the exploratory features pay for a second extraction step and roughly double that; and the lone sampling-based baseline, which repeats generation several times per input, is several times slower than everything else. The practical message is that moving to a richer single-pass configuration is close to free, whereas the sampling route is the only method in the comparison whose cost would bite in a latency-sensitive deployment.

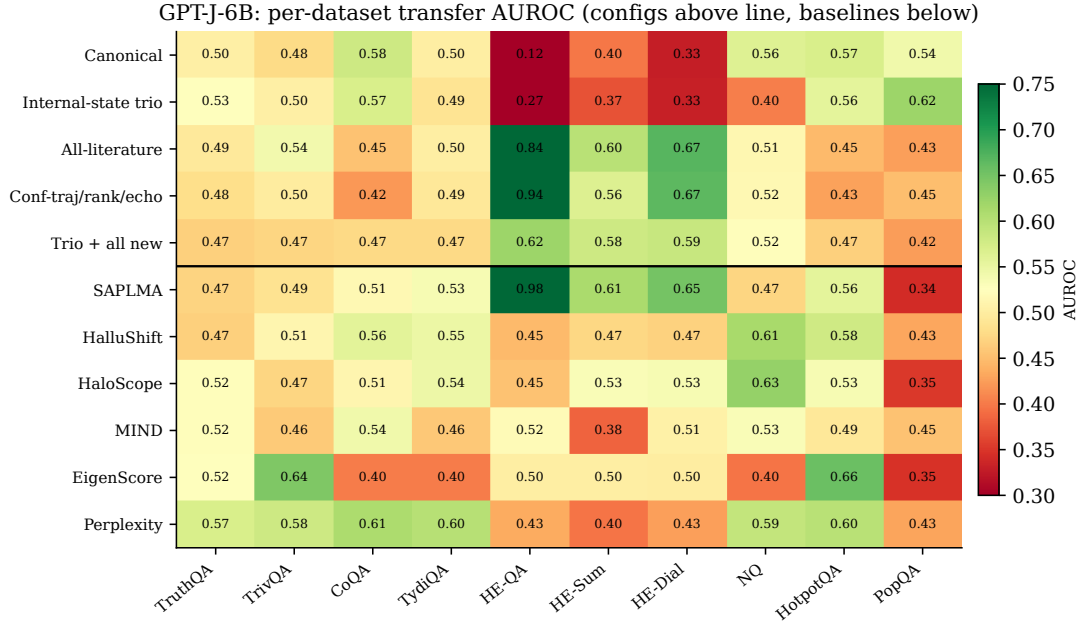


Figure 5.4: Per-benchmark transfer AUROC on the six-billion model, configurations above the line and baselines below. The HaluEval-QA column stands out; elsewhere most cells are near 0.5.

5.8 Discussion

Taken together, the results support a measured set of conclusions rather than a triumphant one.

The internal-state signal is real but, on transfer, weak. In-distribution detection works at a useful level, especially at larger data scales, but transferring a probe trained on encyclopedia continuations to question answering,

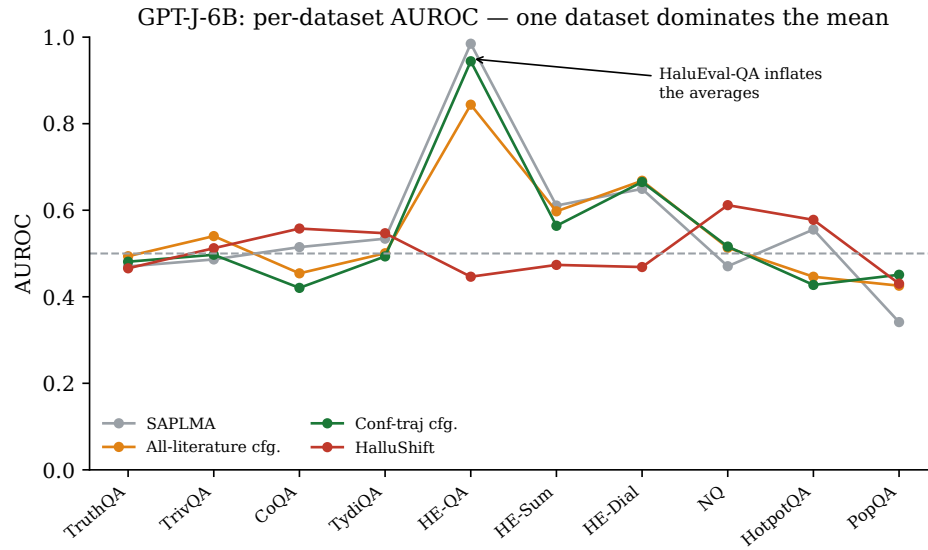


Figure 5.5: The same data as lines. The spike on HaluEval-QA is what separates the leading methods; the other benchmarks cluster near chance.

Table 5.4: Second six-billion-class model (OPT): the four exploratory-feature configurations and the six baselines, by mean transfer AUROC over ten benchmarks. The proposed confidence-trajectory configuration leads every baseline. Higher is better; 0.5 is chance.

Method	Mean transfer (10 benchmarks)
Confidence trajectory + attention rank + prompt echo (C14)	0.551
Unembedding proj. + curvature + confidence (C15)	0.513
Unembedding projection + curvature (C13)	0.482
Internal-state trio + all exploratory (C16)	0.472
Perplexity	0.516
SAPLMA (mid-layer probe)	0.514
MIND (final-layer probe)	0.506
HalluShift	0.486
EigenScore	0.455
HaloScope	0.417

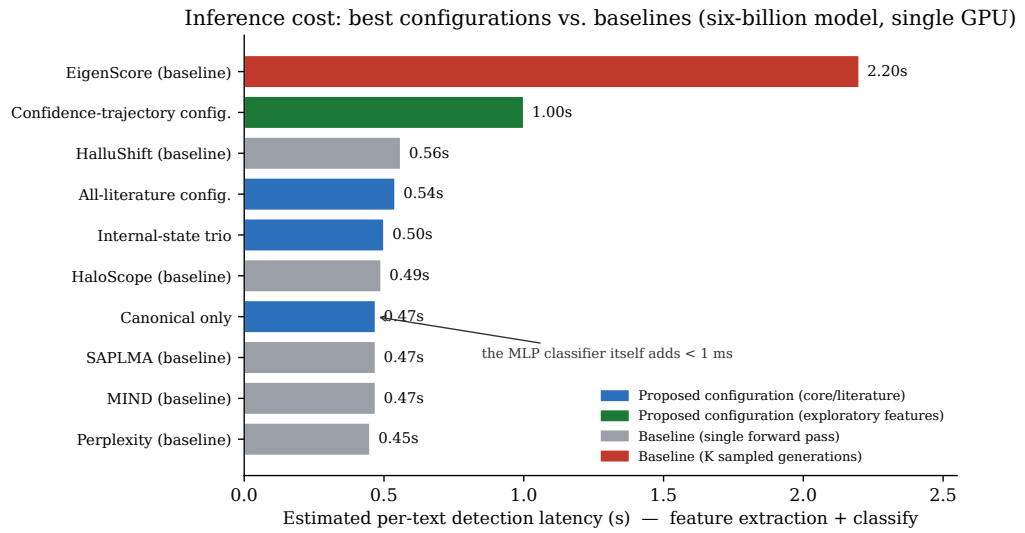


Figure 5.6: Estimated per-text detection latency on the six-billion model (single GPU), derived from the measured feature-extraction stage timings. Single-pass methods are dominated by one forward pass and the classifier adds under a millisecond; the sampling-based baseline repeats generation and is several times slower. The instrumented notebooks record exact per-method timings when run.

Chapter 6

Conclusion and Future Work

6.1 Summary

This dissertation studied whether hallucinations in large language models can be detected cheaply by reading the model’s own internal representations, and which of the many proposed internal signals are worth their cost. We adopted the label-free, per-model data generation of the MIND framework, kept its canonical last-token, last-layer hidden state as a backbone, and added three inexpensive signals from the same forward pass: how much the representation drifts between layers, how much it varies across layers, and how uncertain the model is while generating. We placed these in a controlled ablation alongside literature features and more speculative features, fixed a single classifier and a single evaluation protocol, and compared everything against four established baselines and two simple reference methods on a suite of up to ten benchmarks — all on free cloud hardware.

The honest summary of the findings is threefold. First, detection of held-out in-distribution text works at a useful level, and better at larger data scales, but transferring an encyclopedia-trained probe to question answering, summarisation, and dialogue is hard: on transfer, every method we tested sits close to chance once a single unusually separable benchmark is set aside. Second, on the one model for which we have a complete set of file-backed measurements, the three core features add little on their own, the gains come from richer feature combinations, and even the best configurations match rather than beat the strongest baseline — a supervised probe whose advantage seems to come from reading a better layer rather than from a cleverer feature. Third, the result is model-dependent: on a second model of comparable size a proposed configuration — built from the more exploratory internal-state signals — is instead the strongest method of all, clearing every baseline. Two models of similar size disagreeing about which method wins is itself the finding, and a caution against reading a universal winner off any one model.

The contribution is therefore not a new state-of-the-art detector. It is a unified, free-tier-reproducible comparison of internal-state hallucination features across several open models, built

so that the regime-dependent and model-dependent effects above are visible and checkable rather than hidden inside a single headline number.

6.2 Limitations

We state the limitations plainly, because several of them bound how far the conclusions can be taken.

Two models, not the whole fleet. The comparison rests on two six-billion-parameter models: one with the complete sixteen-configuration, six-baseline sweep, and one with the six baselines measured against the exploratory-feature configurations. The remaining large models have their training data ready but their detection runs are still in progress, so the cross-model claim is drawn from two models rather than the full fleet.

Modest sample sizes. The unified runs use two thousand training examples per model and evaluate on a few hundred rows per benchmark. At this scale, individual AUROCs are noisy, classifiers sometimes collapse to a single class, and small differences between methods should not be over-interpreted. The earlier, larger-scale in-distribution figures suggest that more data helps, but we did not re-run the full ablation at that scale.

Free-tier constraints. The nine-hour session limit, the weekly GPU budget, and the sixteen-gigabyte cards shaped the design: some baselines were dropped for cost, evaluation was capped at a fixed number of rows, and one gated model could not be run at full data size. These are reasonable trade-offs for a reproducible free-hardware study, but they are constraints, and a better-resourced replication might reach steadier numbers.

One benchmark dominates. The transfer averages on the fully measured model are carried by a single easy benchmark. The rankings should be read with that in mind, and per-benchmark results (Appendix A) are more informative than the averages.

Split evaluation on the second model. On the second model the literature and core-feature configurations were scored on the earlier seven-benchmark suite while the exploratory-feature configurations and the baselines used the full ten; a single unified re-run would put all sixteen configurations on the same footing.

6.3 Future work

Several concrete next steps follow from these limitations.

The most immediate is to *complete the sweep*: run the full sixteen-configuration, six-baseline comparison on the second model and on the remaining seven-billion models, and regenerate the gated model’s data at full size. This broadens a two-model comparison into a fleet-wide one and would settle whether the model-dependence seen here is the rule.

The discussion pointed at *layer selection* as the lever most worth pulling. The strongest baseline’s edge came from reading a mid-network layer rather than the final one, so a focused

ablation over the layer at which the canonical and internal-state features are read — on each model — is likely to be more productive than adding further scalar features.

A third direction is *scale and calibration*. Re-running the ablation at the larger data scale used in the mid-term evaluation would test whether the small per-feature effects become reliable, and reporting calibration error and Brier score alongside AUROC would make the detectors more useful as decision-support tools, since a calibrated probability is worth more than a bare verdict.

Finally, the finding that transfer is the hard part suggests studying the *training distribution* itself: mixing a small amount of task-like data into the encyclopedia continuations, or generating continuations from more varied sources, may close part of the in-distribution-to-transfer gap that dominates the present results.

6.4 Closing remark

The modest headline of this work — that inexpensive internal-state features are competitive but not decisive, and that cross-task transfer remains hard — is, we think, more useful than a louder claim would have been. The detectors in this area are often reported under favourable, inconsistent conditions; holding ev

Bibliography

- [1] Sharanya Dasgupta. From vigilance to veracity: Hallucination detection, mitigation, and safety enhancement in large language models (HalluShift). Master’s thesis, Indian Statistical Institute, Kolkata, 2025. Also published at IJCNN 2025.
- [2] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [3] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of EMNLP*, 2023.
- [4] Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it’s lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- [5] Weihang Su, Changyue Wang, Qingyao Ai, Yiran Hu, Zhijing Wu, Yujia Zhou, and Yiqun Liu. Unsupervised real-time hallucination detection based on the internal states of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14379–14391, 2024.
- [6] Xuefeng Du, Chaowei Xiao, and Yixuan Li. HaloScope: Harnessing unlabeled LLM generations for hallucination detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [8] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- [9] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*, 2023.

- [10] Chao Chen, Kai Liu, Ze Chen, Yi Gu, Yue Wu, Mingyuan Tao, Zhihang Fu, and Jieping Ye. INSIDE: LLMs’ internal states retain the power of hallucination detection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [11] Yung-Sung Chuang, Yujia Xie, Hongyin Luo, Yoon Kim, James Glass, and Pengcheng He. DoLa: Decoding by contrasting layers improves factuality in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [12] Yung-Sung Chuang, Linlu Qiu, Cheng-Yu Hsieh, Ranjay Krishna, Yoon Kim, and James R. Glass. Lookback lens: Detecting and mitigating contextual hallucinations in large language models using only attention maps. In *Proceedings of EMNLP*, pages 1419–1436, 2024.
- [13] Mengjia Niu, Hamed Haddadi, and Guansong Pang. Robust hallucination detection in LLMs via adaptive token selection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [14] Yue Zhang et al. D2HScore: Reasoning-aware hallucination detection via semantic breadth and depth analysis in LLMs. *arXiv preprint arXiv:2509.11569*, 2025.
- [15] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a large-scale dataset for fact extraction and verification. In *Proceedings of NAACL-HLT*, 2018.
- [16] Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of EMNLP*, 2023.
- [17] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of ACL*, 2022.
- [18] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of ACL*, 2017.
- [19] Siva Reddy, Danqi Chen, and Christopher D. Manning. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019.
- [20] Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470, 2020.

- [21] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. HaluEval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of EMNLP*, 2023.
- [22] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- [23] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [24] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating the effectiveness of parametric and non-parametric memories. In *Proceedings of ACL*, 2023.
- [25] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Appendix A

Full Result Tables

This appendix gives the complete per-benchmark measurements behind the summary figures and tables of Chapter 5. All numbers are AUROC, read directly from the saved measurement files; 0.5 is chance.

Table A.1 is the full matrix for the six-billion-parameter model that anchors the study: every one of the sixteen configurations and six baselines, scored on the same rows of all ten benchmarks. It is the table from which the means in Section 5.3 are computed, and it makes the per-benchmark spread — in particular the dominance of the HaluEval-QA column — explicit.

Table A.2 is the file-backed diagnostic run for the second six-billion-class model. It covers twelve configurations on the original seven-benchmark suite at a small number of rows per benchmark; as noted in Chapter 5, these numbers are diagnostic rather than scientific, and the extended ten-benchmark run with baselines is awaiting archival.

Table A.1: Full per-benchmark transfer AUROC on the six-billion-parameter model (GPT-J): sixteen feature configurations and six baselines, scored on identical rows of all ten benchmarks. **Bold** marks the best value in each column. Source: the consolidated configuration and baseline measurement files.

Method	TruthQA	TrivQA	CoQA	TydiQA	HE-QA	HE-Sm	HE-DI	NQ	HotQA	PopQA	Avg
C1 canonical	0.503	0.476	0.580	0.502	0.124	0.398	0.330	0.562	0.569	0.541	0.459
C2 +drift	0.492	0.459	0.566	0.492	0.127	0.337	0.339	0.551	0.558	0.553	0.447
C3 +variance	0.478	0.453	0.564	0.497	0.109	0.381	0.349	0.535	0.554	0.534	0.445
C4 +entropy	0.519	0.410	0.541	0.478	0.260	0.389	0.389	0.513	0.524	0.692	0.472
C5 +trio	0.526	0.503	0.570	0.487	0.272	0.371	0.330	0.403	0.557	0.620	0.464
C6 +lookback	0.498	0.483	0.591	0.505	0.068	0.342	0.321	0.544	0.550	0.548	0.445
C7 +logit-lens	0.472	0.482	0.573	0.502	0.079	0.383	0.344	0.535	0.546	0.545	0.446
C8 +margin	0.522	0.478	0.525	0.536	0.434	0.374	0.396	0.558	0.517	0.417	0.476
C9 +l/l/m	0.504	0.511	0.563	0.490	0.126	0.348	0.329	0.455	0.563	0.569	0.446
C10 +all lit.	0.494	0.540	0.454	0.501	0.844	0.597	0.668	0.513	0.446	0.426	0.548
C11 +trio+llm	0.484	0.518	0.546	0.520	0.635	0.391	0.464	0.539	0.603	0.553	0.525
C12 +trio+lit.	0.555	0.534	0.444	0.465	0.408	0.462	0.522	0.484	0.453	0.355	0.468
C13 +urp/curv	0.494	0.507	0.576	0.507	0.065	0.315	0.311	0.572	0.567	0.596	0.451
C14 +conf-trajectory	0.481	0.497	0.421	0.493	0.944	0.564	0.665	0.516	0.427	0.451	0.546
C15 +urp/cv/cf	0.504	0.481	0.440	0.461	0.351	0.527	0.406	0.604	0.449	0.483	0.471
C16 +trio+new	0.465	0.471	0.475	0.475	0.620	0.580	0.585	0.522	0.466	0.424	0.508
<i>Baselines</i>											
SAPLMA	0.470	0.486	0.515	0.534	0.985	0.611	0.649	0.471	0.555	0.341	0.562
Perplexity	0.570	0.580	0.609	0.600	0.434	0.397	0.426	0.589	0.597	0.429	0.523
HalluShift	0.466	0.512	0.558	0.547	0.446	0.474	0.469	0.612	0.578	0.430	0.509
HaloScope	0.523	0.472	0.512	0.545	0.455	0.531	0.535	0.627	0.534	0.352	0.508
MIND	0.522	0.462	0.542	0.458	0.520	0.384	0.505	0.532	0.493	0.453	0.487
EigenScore	0.523	0.641	0.400	0.402	0.501	0.500	0.500	0.395	0.656	0.347	0.487

Table A.2: Diagnostic configuration run on the second six-billion-class model (OPT-6.7B), the part backed by a measurement file in the repository: twelve configurations on the original seven-benchmark suite at a small number of rows per benchmark. At this sample size the numbers are diagnostic, not scientific. AUROC; 0.5 is chance.

Cfg.	TruthQA	TrivQA	CoQA	TydiQA	HE-QA	HE-Sm	HE-DI	Avg
C1 canonical	0.328	0.535	0.571	0.503	0.054	0.423	0.345	0.394
C2 +drift	0.338	0.567	0.564	0.523	0.039	0.316	0.313	0.380
C3 +variance	0.308	0.561	0.550	0.537	0.025	0.321	0.310	0.373
C4 +entropy	0.303	0.539	0.556	0.542	0.126	0.342	0.336	0.392
C5 +trio	0.276	0.485	0.502	0.492	0.228	0.380	0.342	0.386
C6 +lookback	0.286	0.546	0.558	0.506	0.032	0.361	0.316	0.372
C7 +logit-lens	0.313	0.527	0.542	0.514	0.026	0.336	0.309	0.367
C8 +margin	0.308	0.528	0.559	0.516	0.037	0.335	0.313	0.371
C9 +l/l/m	0.303	0.519	0.551	0.504	0.055	0.334	0.349	0.374
C10 +all lit.	0.308	0.626	0.562	0.567	0.246	0.395	0.353	0.437
C11 +trio+llm	0.328	0.553	0.551	0.538	0.049	0.286	0.302	0.373
C12 +trio+lit.	0.293	0.528	0.554	0.498	0.020	0.352	0.318	0.366

Appendix B

Hyperparameters and Implementation Details

For reproducibility, this appendix records the settings used throughout. All runs use a fixed random seed of 42.

Data generation

- Source text: encyclopedia articles; the first sentences are retained and a named entity occurring after the first sentence is selected.
- Faithful/hallucinated decision: the original entity’s first token must appear among the model’s top four predicted tokens to be labelled faithful.
- Generation cap during labelling: a short continuation of up to 48 new tokens.
- Scale: one thousand examples per class (two thousand total) for the large models; smaller counts for the iteration and smoke models, as recorded in Chapter 5.
- Numeric format: bfloat16, with no further quantisation.

Classifier

- Architecture: a five-layer perceptron with progressively narrowing hidden widths, rectified-linear activations, and a two-way softmax output.
- Loss and optimiser: cross-entropy; Adam with learning rate 5×10^{-4} and weight decay 10^{-5} .
- Batch size 32; ten training epochs.
- Features standardised by a scaler fitted on the training split only.
- Split: 80% train / 20% test, identical across all configurations.

Evaluation

- Each method — configuration or baseline — is scored on the same deterministic first- N slice of each benchmark, with N up to 350 rows per benchmark in the unified runs.
- Metrics: AUROC (primary), accuracy, precision, recall, F1; calibration (Brier score, expected calibration error) where the measurement file records it.
- In-distribution and transfer numbers are never averaged together.

Baseline settings (six-billion model)

- Supervised mid-layer probe: hidden state read at an intermediate layer (layer 17 of 28).
- Logit-lens divergence feature: early/late comparison taken at layer 14.
- Spectral baseline: best layer and threshold selected on a validation split.
- Sampling-based score: a small number of stochastic samples per query, with a ridge term for numerical stability.

Hardware

- Primary: a free cloud notebook with two sixteen-gigabyte T4 GPUs (thirty-two gigabytes combined), nine-hour session cap, thirty GPU-hours per week.
- Fallback: a free single-T4 notebook for the smaller models.
- The seven-billion models require both GPUs; a single sixteen-gigabyte card cannot hold a seven-billion model in bfloat16 together with generation memory.

Appendix C

Reproducibility Notes

The study is designed to be reproducible on a free account. Three points are worth emphasising. First, the training data is generated separately for each model, because probes do not transfer between models; a detector is therefore always trained on data from the same model it monitors. Second, every configuration and baseline within a model shares the same generated data, the same train/test split and seed, and the same evaluation rows, so that differences in measured performance are attributable to the features rather than to incidental differences in setup. Third, the evaluation benchmarks are downloaded once and cached so that runs are not dependent on network conditions, and the per-benchmark row count is fixed so that cost stays within the free-tier budget. Where a model's measurement file is not yet archived, its numbers are marked as pending in Chapter 5 rather than reported as results.